

Docker, Pourquoi et comment ?

1. pourquoi utiliser Docker ?

1.1 Le problème que Docker résout : ça marche sur ma machine

L'expression **ça marche sur ma machine** : veut dire que l'application fonctionne sur l'ordinateur du développeur, mais ne fonctionne plus ailleurs (sur un serveur ou sur la machine d'un collègue). Cela arrive parce que **l'environnement n'est pas le même** : versions différentes, dépendances manquantes, configuration qui change.

Docker résout ce problème :

- il **emballe l'application + toutes ses dépendances** dans une **image**,
- cette image tourne dans un **conteneur**,
- le conteneur fonctionne **exactement pareil** sur n'importe quelle machine où Docker est installé.

En résumé :

Docker garantit que l'application fonctionne partout de la même manière, en supprimant les différences d'environnement.

Sources utilisées :

- Docker Docs (source officielle) : [Docker Docs](#)
- Dev.to – [Docker pour Débutants en 2026 : Guide Complet - DEV Community](#), Lucas M. DevDev.
- Blog Stéphane Robert – [Tutoriel Docker : le guide pour apprendre de A à Z](#)

1.2 Situer Docker par rapport à :

A. Docker vs environnement virtuel Python

Un **environnement virtuel Python** isole seulement le monde Python : l'interpréteur et les bibliothèques Python. Il ne touche pas au système d'exploitation, aux bibliothèques système, ni aux services externes. Donc, même si deux développeurs utilisent un virtualenv, leurs machines peuvent rester différentes, et l'application peut fonctionner chez l'un mais pas chez l'autre.

Les sources expliquent que **Docker va beaucoup plus loin** : un conteneur embarque l'ensemble du système utilisateur, toutes les dépendances, la configuration, et même la version exacte de Python. Grâce à cela, l'environnement est identique partout, ce qui supprime les différences entre machines. L'article fr.python-3.com insiste sur ce point : un virtualenv encapsule uniquement Python, alors qu'un conteneur encapsule tout le système.

B. Docker vs machine virtuelle (VM)

Une machine virtuelle est une « **fausse machine** » complète, avec son propre système d'exploitation, ses ressources dédiées et une isolation très forte. Les VM sont **plus lourdes, plus lentes** à démarrer et consomment **plus** de CPU et de RAM.

Docker est plus léger : les conteneurs partagent le noyau de l'hôte, démarrent **très vite** et consomment **moins de ressources**. Les conteneurs sont donc plus adaptés pour exécuter des applications **rapidement**, sans recréer une machine entière.

En résumé, Docker se situe **entre** le **venv** et la **VM** : plus complet qu'un environnement Python, mais beaucoup plus léger qu'une machine virtuelle.

C. Docker est justifié dès qu'on veut un environnement **reproductible**, portable et facile à déployer ; il devient de **l'overengineering** pour un petit script local ou une application simple qui n'a pas besoin d'être conteneurisée.

Sources utilisées :

- fr.python-3.com : [Quelle est la différence entre Docker et Python virtualenv ?](#)
- AWS : [Qu'est-ce qu'une machine virtuelle ? – Explication de la machine virtuelle – AWS](#)
- AWS : [Docker vs VM - différence entre les technologies de déploiement d'applications - AWS](#)
- Oracle : [Qu'est-ce qu'une machine virtuelle \(VM\) Définition machine virtuelle | Oracle France](#)
- Codnum : [Docker vs machine virtuelle : quelles différences ?](#)
- Freelance-Informatique : [Docker vs VM : quelles différences pour les projets modernes ? | Les actualités du Freelance](#)
- Docker Docs : [Docker Docs](#)
- Okoone : [6 raisons pour lesquelles les développeurs ne jurent que par les conteneurs Docker et OCI | Okoone](#)
- Hosting : [15 cas d'utilisation de Docker à connaître](#)

2. Comment utiliser Docker ?

2.1 Une image Docker :

Une **image Docker** est un modèle qui contient tout ce qu'il faut pour lancer une application : le code, l'exécution, les variables d'environnement, les bibliothèques et les fichiers de configuration. c'est un paquet immuable composé de **couches**, qui sert de base pour créer un conteneur.

- Exemple :

Si je veux exécuter une application Python, je peux utiliser l'image officielle **python**. Elle contient déjà Python, les bibliothèques nécessaires et la configuration de base. Comme l'image est immuable et composée de couches, elle permet de lancer la même application partout de la même manière.

- devopssec.fr : [Fonctionnement et manipulation des images Docker](#)
- docker.ubitoools.com : [Qu'est-ce qu'une image ? | Docker Docs](#)

2.2 Un conteneur :

Un conteneur est une image Docker en cours d'exécution. C'est l'application qui tourne réellement, isolée du reste du système. Les articles expliquent qu'un conteneur est léger, démarre très vite et contient exactement l'environnement défini dans l'image. Il se comporte toujours de la même manière, peu importe la machine où il est lancé.

Exemple : si je lance l'image **python**, le conteneur démarre un environnement Python prêt à exécuter mon script, avec les mêmes versions et dépendances partout. Le conteneur est isolé du reste du système et démarre très vite, car il utilise exactement l'environnement défini dans l'image.

- IT-Connect : [Qu'est-ce que Docker et pourquoi l'utiliser ?](#)
- Seewide Consulting : [Docker en 2026 : guide complet de la conteneurisation pour les équipes de développement | Seewide Consulting](#)
- GitLab : [Qu'est-ce que Docker ? Fonctionnement, avantages, limites](#)
- Coldwa.st : [Docker, c'est quoi ? Les conteneurs expliqués clairement \(2026\)](#)

2.3 Un Dockerfile

Un Dockerfile est un fichier texte qui décrit comment construire une image Docker. Il contient des instructions simples comme choisir une image de base, copier des fichiers, installer des dépendances ou définir une commande de démarrage. Les sources montrent qu'un Dockerfile permet de créer des images reproductibles et faciles à maintenir.

Exemple : un Dockerfile peut commencer par « **FROM python:3.12** », pour choisir une image de base. Ensuite, il peut copier mon code avec **COPY . /app** et installer mes dépendances avec « **pip install -r requirements.txt** ». Chaque instruction crée une couche dans l'image, ce qui permet à Docker de réutiliser les couches déjà construites si elles n'ont pas changé.

- Stéphane Robert : [Guide Dockerfile : Créer des Images Docker Optimisées](#)

- DevOps Aibit : [Mastering Dockerfile layer caching](#)
- Dockerpros : [Cache Docker - Dockerpros](#)

2.4 Les couches et le cache

Une image Docker est construite en plusieurs couches. Chaque instruction du Dockerfile crée une nouvelle couche. Docker garde ces couches en cache pour accélérer les constructions : si une couche n'a pas changé, Docker la réutilise. Les articles expliquent que cela permet des builds beaucoup plus rapides.

Exemple : si mon Dockerfile installe Python dans une première couche, puis copie mon code dans une deuxième couche, Docker garde la couche Python en cache. Quand je modifie seulement mon script, Docker reconstruit uniquement la couche du code et réutilise la couche Python depuis le cache, ce qui accélère le build.

- Dockerpros – [Cache Docker - Dockerpros](#)
- DevOps Aibit – [Mastering Dockerfile layer caching](#)
- Stéphane Robert – [Guide Dockerfile : Créer des Images Docker Optimisées](#)
- Docker Docs - [Docker build cache | Docker Docs](#)

2.5 Un registre (Docker Hub, GitLab Registry, etc.)

Un registre est un endroit où les images Docker sont stockées et partagées. Docker Hub est le registre le plus connu, mais il existe aussi GitLab Registry ou des registres privés. Les sources montrent que les registres permettent de télécharger des images officielles ou de publier ses propres images pour les déployer ailleurs.

Exemple : je peux pousser mon image « **monapp:1.0** » sur Docker Hub, puis la récupérer sur un serveur avec un simple « **pull** » pour déployer exactement la même version de l'application.

- RDR-IT : [Le Docker Hub et les registres](#)
- IT-Connect : [Prise en main du Docker Hub, des versions et tags d'images](#)
- Docker Docs : [registry - Official Image | Docker Hub](#)

2.6 Un volume

Un volume est un espace de stockage persistant utilisé par un conteneur. Les articles expliquent que les données d'un conteneur disparaissent quand il s'arrête, donc les volumes servent à conserver les fichiers importants (bases de données, logs, uploads). Un volume permet aussi de partager des données entre plusieurs conteneurs.

Exemple : si j'ai un conteneur MySQL, je crée un volume pour que les données de la base soient conservées même si le conteneur est supprimé. Le volume est stocké en dehors du conteneur, ce qui protège les fichiers importants.

- Alphorm : [Comprendre et Gérer les Volumes Docker : Types et Bonnes Pratiques](#)
- DevOpsSec : [Fonctionnement et manipulation des volumes dans Docker](#)

2.7 Un port

Un port est un point d'entrée pour accéder à une application dans un conteneur. Les sources expliquent qu'un conteneur est isolé, donc il faut "publier" un port pour rendre l'application accessible depuis l'extérieur. On utilise souvent la syntaxe « **-p 8080:80** » pour relier un port de la machine au port du conteneur.

Exemple : si mon conteneur exécute un serveur web sur le port **80**, je publie le port **8080** pour y accéder via **localhost:8080**. Comme le conteneur est isolé, la publication du port est nécessaire pour communiquer avec l'extérieur.

- DataCamp : [Comment exposer un port Docker | DataCamp](#)
- UnixLinux.online : [Comment exposer ou publier le port Docker](#)
- Dynops : [5. Les ports - Docker](#)

2.8 Docker Compose

Docker Compose est un outil qui permet de lancer plusieurs conteneurs ensemble, comme une application complète. Les articles expliquent qu'on décrit les services dans un fichier **docker-compose.yml**, puis on démarre tout avec une seule commande. Compose est très utile pour les projets qui ont plusieurs composants (API, base de données, front-end).

Exemple : je peux définir un service **web** et un service **base de données** dans un fichier Compose, puis lancer les deux avec "**docker compose up**".

- **Docker Docs :** [Docker Compose | Docker Docs](#)

- **FreeCodeCamp :** [What is Docker Compose? How to Use it with an Example](#)

3. première manipulation

J'ai ouvert la Powershell et Docker desktop tourne.

J'ai utilisé la commande "**docker pull hello-world**" pour récupérer une image depuis le registre **Docker Hub**.

Puis j'ai lancé cette image avec "**docker run hello-world**", ce qui a créé un conteneur. Le conteneur s'est exécuté, a affiché un message de test qui explique ce que Docker vient de faire, puis s'est arrêté.

J'ai vérifié l'historique des conteneurs avec "**docker ps -a**".

j'ai observé le résultat : un conteneur nommé **funny_hugle**, basé sur l'image **hello-world** avec l'ID **5ca8a64651c6**, créé il y a **une minute** et dont le statut est **Exited (0)**, et sans **ports**, c'est normal, car l'image **hello-world** n'expose aucun port : elle ne lance pas de service accessible depuis l'extérieur.

Le conteneur exécute simplement un programme qui affiche un message, puis s'arrête, donc Docker n'a pas besoin de publier de port.

Cette manipulation m'a permis de voir concrètement la différence entre une image (modèle) et un conteneur (instance en cours d'exécution).

4. Retour à l'observatoire

Pour conteneuriser mon observatoire, je suivrais le même cycle que celui présenté dans le schéma du livre **Docker Deep Dive – Zero to Docker in a Single Book** : **Build** → **Ship** → **Run**.

Je commencerais par créer une **image** Docker qui contient tout le pipeline : le code Python, les dépendances, les fichiers de configuration et l'environnement nécessaire pour exécuter les traitements. Cette image serait construite avec un **Dockerfile**, où chaque instruction crée une **couche**. Docker utiliserait le cache pour reconstruire seulement les couches modifiées, ce qui rend les mises à jour plus rapides.

C'est la partie **Build** du schéma.

Une fois l'image prête, je pourrais la publier dans un registre comme **Docker Hub** ou un registre **privé**. Cela permettrait de récupérer facilement la même image sur n'importe quelle machine.

C'est la partie **Ship** du schéma.

À partir de cette image, Docker créerait un conteneur qui exécute réellement le pipeline, toujours de la même manière. Pour organiser l'observatoire, j'utiliserais **Docker Compose**. Je définirais **deux services** : un service pour le **pipeline Python** et un service pour la **base de données PostgreSQL**. Compose permet de lancer les deux conteneurs **ensemble** et de les connecter automatiquement. Le pipeline pourrait envoyer les données directement dans PostgreSQL.

C'est la partie **Run** du schéma.

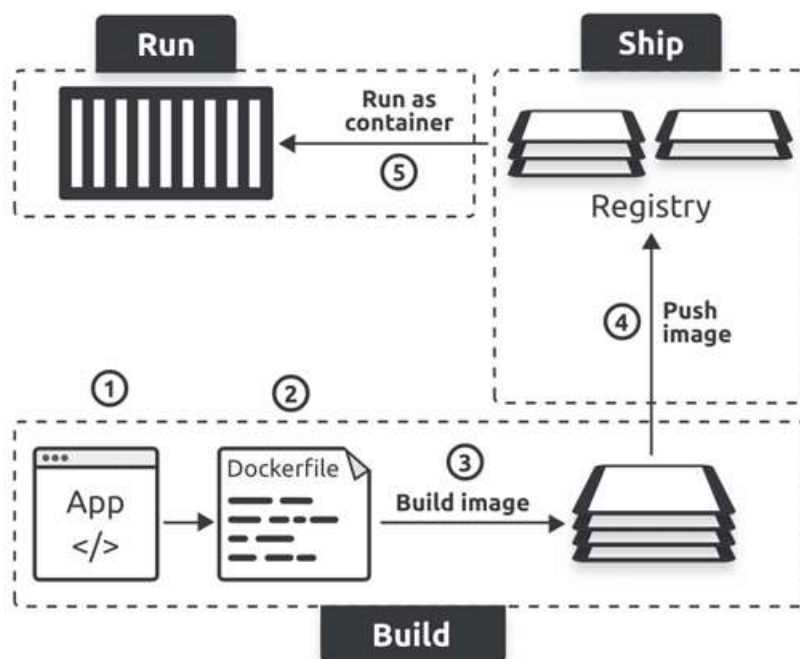
Pour ne pas perdre les données, je placerais un **volume** sur PostgreSQL. Le volume permet de conserver les fichiers de la base même si le conteneur est supprimé ou redémarré. Si un jour le pipeline expose une API ou une interface, je pourrais publier un port pour y accéder depuis l'extérieur.

Cette organisation rendrait l'observatoire reproductible, portable et facile à déployer : l'image contient le pipeline, Compose orchestre les services, et le volume protège les données. Le schéma du livre représente exactement ce processus : construire l'image, la stocker dans un registre, puis l'exécuter comme conteneur dans une stack Docker.

- Schéma de conteneurisation de l'observatoire :

Ressource : Nigel Poulton, [Docker Deep Dive](#) .

Schéma "Build → Ship → Run", p. 9



Merci

Marie Antoinette KILO

kilomarinette@gmail.com